Pipeline ETL para Agricultura de Precisão

Integração de Sistemas de Informação 2026/27

Diogo Rocha · n.º 29339

Licenciatura em Engenharia de Sistemas Informáticos

1. Enquadramento

Este relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação (ISI), do curso de Licenciatura em Engenharia de Sistemas Informáticos, ano letivo 2026/27.

- **Equipa:** Individual
- **Unidade curricular:** Integração de Sistemas de Informação
- **Curso:** Licenciatura em Engenharia de Sistemas Informáticos
- **Docentes:** Luís Ferreira e Óscar Ribeiro
- **Plataforma:** Pentaho Data Integration (Kettle) 10.2
- **Repositório Git:** https://github.com/Diogo193/ETL_agriculture.git

2. Problema

Numa exploração agrícola com várias culturas (milho, vinha e olival), sensores de solo enviam leituras de humidade e temperatura. Cada cultura tem um intervalo de humidade considerado adequado, e o agricultor precisa de saber quando a humidade média de uma cultura desce abaixo do seu limite, ou seja, quando é necessário regar.

Os dados reais têm problemas que um processo ETL tem de resolver:

- leituras duplicadas (mesmo sensor e mesmo instante);
- identificadores de sensor inválidos, timestamps impossíveis (mês 34, minuto 89), culturas desconhecidas e valores nulos;
- necessidade de cruzar as leituras com os parâmetros de cada cultura, guardados noutra fonte e noutro formato;
- necessidade de ajustar o limiar de alerta às condições meteorológicas (calor e vento aumentam a perda de água);
- necessidade de partilhar dados com um destinatário externo sem expor os identificadores reais dos sensores.

O objetivo do trabalho é construir um pipeline ETL completo, orquestrado por um Job, que recolhe estes dados, os valida e transforma, os carrega numa base de dados e envia alertas por e-mail.

3. Estratégia utilizada

O processo segue as três fases pedidas no enunciado, cada uma implementada numa transformação Kettle, orquestradas pelo Job `jr_program`.

Fase	Transformação	Função
E – Extração	`tr_extract`	Recolhe leituras, parâmetros das culturas e meteorologia
T – Transformação	`tr_transform`	Valida, limpa, calcula, agrega e anonimiza
L – Carregamento	`tr_load`	Persiste na base de dados, gera alertas e ficheiros

Fontes e formatos

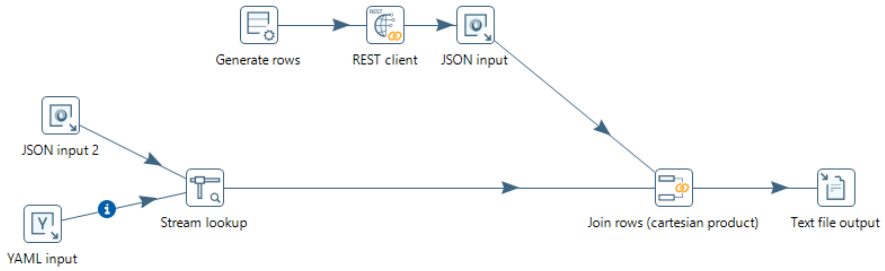
Origem	Formato	Utilização
`data/input/sensores.json`	JSON	Leituras dos sensores
`data/input/culturas.yaml`	YAML	Humidade mínima e máxima por cultura
API REST Open-Meteo	JSON	Temperatura do ar e velocidade do vento atuais (latitude 41.53, longitude -8.62)

Destinos e formatos de saída: CSV (`dados_brutos`, `dados_processados`, `rejeitados`), XML (`dados_anon`), texto (`alertas.txt`), base de dados SQLite (`agricola.db`, tabela `relatorio_humidade`), log de execução e e-mail.

Operadores principais: Generate rows, REST client, JSON input, YAML input, Stream lookup, Join rows, Regex evaluation, Filter rows, Add constants, Calculator, Formula, Sort rows, Unique rows, Memory group by, Get variables, Add a checksum, Select values, XML output, Table output, Write to log, Text file output; no Job: Set variables, Simple evaluation, Abort job e Mail.

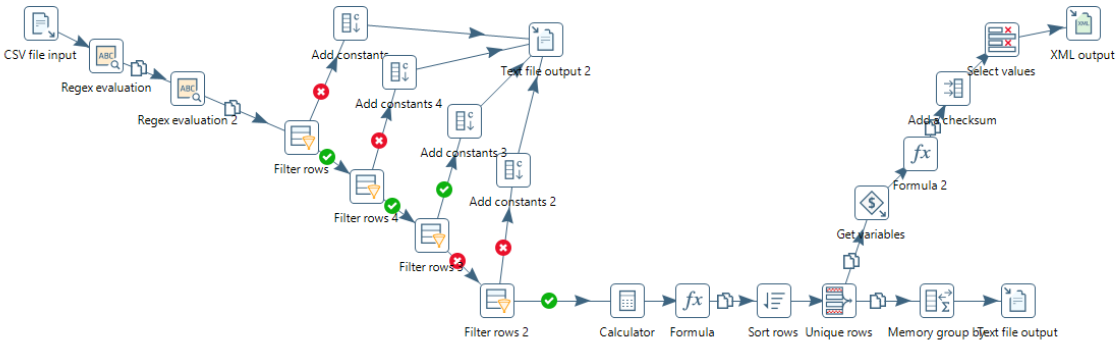
4. Transformações

4.1 tr_extract (Extração)



1. **Generate rows** produz uma linha que dispara a chamada à API.
2. **REST client** faz um GET ao Open-Meteo e **JSON input** extrai `temperatura_ar` (`$.current_weather.temperature`) e `velocidade_vento` (`$.current_weather.windspeed`).
3. **JSON input 2** lê as leituras de `sensores.json` (`sensor_id`, `cultura`, `humidade`, `temperatura_solo`, `timestamp`).
4. **YAML input** lê `culturas.yaml`; o **Stream lookup** junta a cada leitura a `humidade_minima` e `humidade_maxima` da respetiva cultura.
5. **Join rows (cartesian product)** anexa os dados meteorológicos (uma linha) a todas as leituras.
6. **Text file output** grava o resultado em `dados_brutos.csv`.

4.2 tr_transform (Transformação)



Validação (Regex e filtros em série). Cada leitura passa por filtros sucessivos; a que falhar é enviada para `rejeitados.csv` com uma coluna `motivo`, em vez de se perder em silêncio.

Passo	Regra	Motivo se falhar
Regex evaluation	`sensor_id` corresponde a `^SEN-\d{3}$`	ID inválido
Regex evaluation 2	`timestamp` em formato ISO UTC com mês 01–12, dia 01–31, hora 00–23, minutos e segundos 00–59, terminado em Z	timestamp inválido
Filter rows 3	`cultura` não começa por ERR	cultura desconhecida
Filter rows 2	`humidade` não nula	humidade nula

Cálculo. O **Calculator** calcula `deficit_humidade = humidade_minima - humidade`. O **Formula** calcula o limiar de alerta ajustado ao tempo: `limiar_alerta = humidade_minima + 5 (se temperatura_ar > 30) + 3 (se velocidade_vento > 20)`.

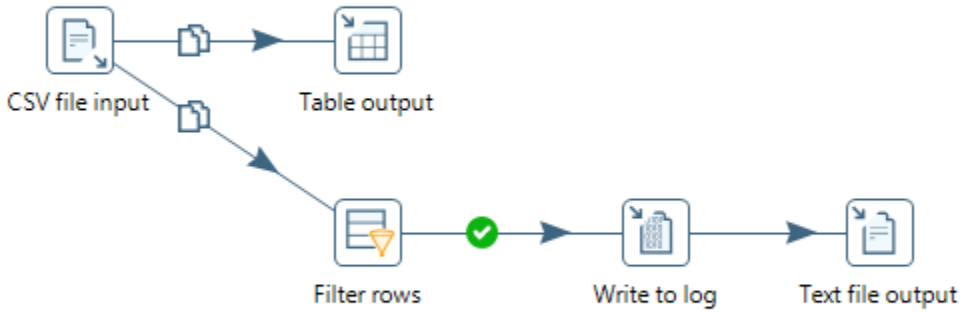
Ordenação e duplicados. Sort rows ordena por `sensor_id` e `timestamp`; **Unique rows** remove duplicados com a chave `sensor_id + timestamp`. Uma leitura do mesmo sensor noutro instante é mantida.

Bifurcação. Depois da deduplicação o fluxo divide-se em dois ramos:

- **Ramo interno (agricultor e técnico): Memory group by** agrupa por `cultura` e `limiar_alerta` e calcula a média de humidade; **Text file output** grava `dados_processados.csv`.
- **Ramo externo (anonimizado): Get variables** obtém o `SALT`; **Formula 2** concatena `sensor_id` com o salt; **Add a checksum** aplica SHA-256 e produz `sensor_anon`; **Select**

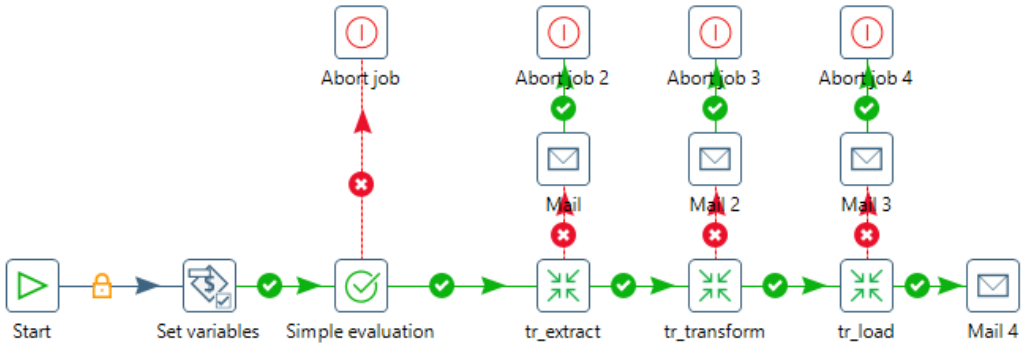
values escolhe os campos a partilhar; **XML output** grava `dados_anon.xml`. O hash é irreversível e o salt impede a recuperação do ID por tentativa de todos os valores possíveis.

4.3 tr_load (Carregamento)



1. **CSV file input** lê `dados_processados.csv`.
2. **Table output** grava em `relatorio_humidade` (SQLite) com *truncate*, para que cada execução substitua os dados anteriores em vez de acumular linhas duplicadas.
3. **Filter rows** seleciona as culturas com `media_humidade < limiar_alerta`; **Write to log** regista o alerta; **Text file output** grava `alertas.txt`.

5. Jobs



O Job `jr_program` orquestra todo o processo:

1. **Start** → **Set variables** define o `SALT`.
2. **Simple evaluation** valida o salt com a expressão regular `^(?!\$\{).+$` (rejeita um valor por definir). Se falhar, **Abort job** pára o processo antes de qualquer transformação.
3. Em sequência: `tr_extract` → `tr_transform` → `tr_load`, cada ligação apenas em caso de sucesso.
4. **Tratamento de falha:** se uma transformação falhar, é enviado um e-mail específico (Extract, Transform ou Carregamento falhou) e o Job termina com **Abort job** e mensagem própria.
5. **Sucesso:** após o `tr_load`, o passo **Mail 4** envia o alerta de rega com `alertas.txt` em anexo.
6. **Log:** cada transformação regista no ficheiro `dataintegration/logs/job_log.log` (modo *append*).

Os e-mails foram testados com um servidor SMTP local de teste (smtp4dev).

6. Resultados

Com os dados de teste, as 11 leituras de `sensores.json` produziram:

- 3 leituras rejeitadas (SEN-003 por cultura desconhecida; SEN-005 e SEN-006 por timestamp inválido), registadas em `rejeitados.csv`;
- 1 duplicado removido (SEN-001 com o mesmo timestamp); 7 leituras válidas seguem para agregação e para o ficheiro anonimizado;
- resultado por cultura:

Cultura	Limiar de alerta	Humidade média	Alerta
MILHO	60	53,3	Sim
VINHA	40	46,5	Não
OLIVAL	30	39	Não

À data dos testes, a temperatura do ar (15,8) e o vento (11,2) estavam abaixo dos limites de ajuste, pelo que os limiares coincidem com a humidade mínima de cada cultura.

7. Vídeo de demonstração

https://drive.google.com/file/d/1wr-2wHoVuEphqnKi2UwvQj_g_BjHxob_/view?usp=sharing

8. Conclusão e trabalhos futuros

O trabalho cobre as três fases do ETL com três formatos de entrada (JSON, YAML, API REST), validação com expressões regulares, cruzamento de dados, agregação, ordenação, remoção de duplicados, anonimização, exportação em XML e CSV, persistência em SQLite, geração de logs e orquestração por Job com tratamento de falhas e notificação por e-mail.

Limitações: o salt está definido diretamente no Job (devia vir de um parâmetro ou segredo externo); os e-mails usam um servidor de teste local; as leituras vêm de um ficheiro estático e não de um fluxo contínuo; os incrementos do limiar (5 e 3 pontos) são valores fixos.

Trabalhos futuros: fonte em streaming com Node-RED e MQTT; dashboard com as médias por cultura e alertas; exportação por SFTP; leitura de leituras a partir de uma base de dados; relatório em PDF; execução agendada.

9. Referências

- Pentaho Data Integration (Kettle) – documentação oficial https://docs.pentaho.com/pdia-data-integration
- Open-Meteo – API de previsão meteorológica: https://open-meteo.com/
- SQLite: https://www.sqlite.org/
- smtp4dev: https://github.com/rnwood/smtp4dev
- Enunciado do TP1 de ISI 2026/27 (Luís Ferreira e Óscar Ribeiro)